.NET BACKEND DEVELOPMENT

Self-Study Progress Report — Part II

Database Migration, EF Core Integration & Async CRUD Operations

Prepared by	Dereje Abtew
Position	Application Management Officer (Trainee Banker)
Report Date	April 30, 2026
Study Reference	Julio Casal — .NET REST API Tutorial (YouTube)
Continuation of	Part I Report — April 27, 2026

1. Introduction

This report is a continuation of my .NET backend development self-study progress report submitted on April 27, 2026. In this second phase of my learning journey, I have advanced beyond the foundational REST API concepts into more advanced and critical areas of enterprise backend development. Specifically, this report covers database migration using Entity Framework Core, connection string management and environment-based configuration, asynchronous programming, database-backed CRUD operations, data seeding, logging configuration, and the implementation of a Genres API endpoint.

The content documented here reflects hands-on study sessions conducted over multiple days, following the tutorial series by Julio Casal on YouTube. Each concept has been applied directly to the GameStore.Api project created in Part I.

2. Key Concepts Studied

Before documenting the implementation steps, the following table summarizes the new concepts I studied and understood during this phase of training:

Concept	Explanation
Entity vs. DTO	An Entity (Model) contains all the raw data as stored in the database table — no filtering. A DTO (Data Transfer Object) contains only the

Concept	Explanation
	fields that are safe and relevant to expose to the client. The Model determines what columns exist in the database table; the DTO determines what the API returns to the user.
Database Migration	A migration is a version-controlled script that tells EF Core how to create or modify the database schema. Running 'migrations add' generates the script; 'database update' applies it to the actual database file.
Connection String	A connection string is a configuration value that tells the application where to find and how to connect to the database. It is stored in appsettings.json and referenced in the application code via the Configuration API.
Environment Variables	Environment variables allow sensitive configuration (such as production database paths) to be stored outside of source code files, improving security and flexibility across different deployment environments.
Data Seeding	Data seeding is the practice of pre-populating a database with initial data — such as default genres or categories — so that the application is not empty when first launched.
Logging Levels	Logging levels control how much information the application records about its own operations. Levels range from Trace (most detailed) to Critical (most severe). Proper logging configuration prevents log files from becoming bloated while still capturing meaningful events.
Dependency Injection	Dependency Injection (DI) is a design pattern where required services (such as the database context) are automatically provided to endpoint handlers by the .NET runtime, rather than being manually instantiated.
Asynchronous Programming	Async/await allows the application to handle multiple client requests concurrently. When one request is waiting for a database response, the thread is released to handle other requests — greatly improving scalability and performance.

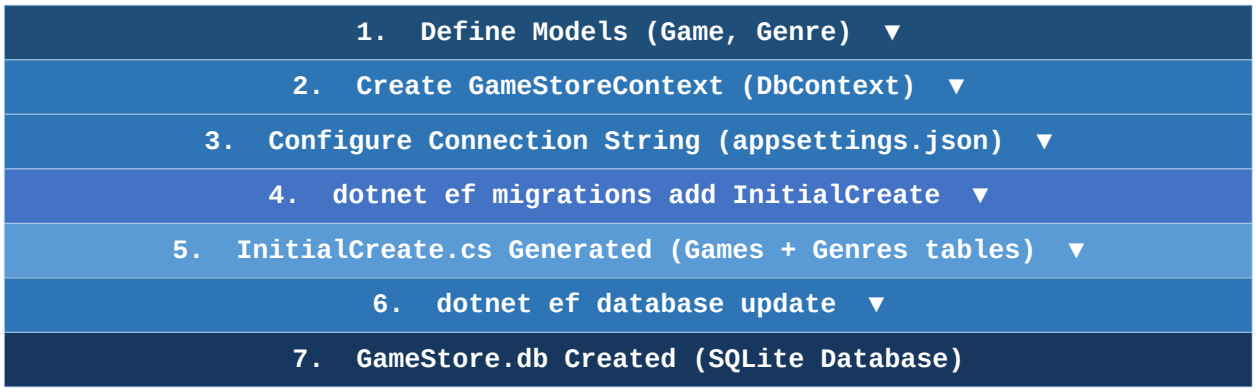
3. Database Migration

3.1 Installing EF Core Tooling

To begin the database migration process, I first installed the dotnet ef global tool, which provides command-line access to Entity Framework Core migration commands. I also installed the Microsoft.EntityFrameworkCore.Design NuGet package through Solution Explorer, as this package is required by the migration tooling to inspect the application's models and context at design time.

3.2 Migration Flow Diagram

The diagram below illustrates the complete database migration workflow, from model definition through to the creation of the physical SQLite database file:



3.3 Running the Migration Command

After installing the required tooling, I executed the following command in the terminal to generate the initial migration:

```
dotnet ef migrations add InitialCreate --output-dir Data/Migrations
```

Upon successful execution, Entity Framework Core analyzed my Game and Genre model classes and generated a migration file named InitialCreate.cs inside the Data/Migrations folder. This file contains the instructions to create two database tables: Games and Genres, with all the columns and relationships defined in my models.

3.4 Applying the Migration

To apply the migration and physically create the database, I ran:

```
dotnet ef database update
```

> ⚠️ **Issue Encountered: Connection String Syntax Error**
>
> When executing 'dotnet ef database update', I encountered an error caused by an incorrect connection string syntax in appsettings.json. The SQLite connection string was not formatted correctly, which prevented EF Core from locating the target database file. This issue was identified, and the correct syntax was applied in the subsequent study session as documented in Section 4 below.

4. Connection String Configuration

4.1 Defining the Connection String

To resolve the migration error, I correctly defined the database connection string in the appsettings.json configuration file using the following format:

```
"connectionStrings": {
  "GameStore": "Data Source=GameStore.db"
}
```

I then updated the DataExtensions.cs file to read this connection string from the application's configuration system using the standard .NET Configuration API:

```
var connString = builder.Configuration.GetConnectionString("GameStore");
```

This approach follows the recommended .NET pattern of separating configuration from code, making the application easier to manage and deploy across different environments.

4.2 Environment-Based Configuration

To demonstrate how production environments can use different database configurations without modifying source code, I created an environment variable using the PowerShell command:

```
$env:ConnectionStrings__GameStore="Data Source=Production.db"
```

When the application starts with this environment variable set, EF Core automatically uses Production.db as the target database, overriding the appsettings.json value. This is a critical concept in real-world application deployment, where development, staging, and production environments each use separate databases.

> ℹ️ **Observed Behavior: WAL and SHM Files**
>
> I observed that when the Production.db file was deleted from File Explorer and the application was restarted, the database file was not immediately recreated. Instead, SQLite created companion files with the extensions -wal (Write-Ahead Log) and -shm (Shared Memory) belonging to the base database. This behavior is related to SQLite's WAL journaling mode, which keeps write operations in a separate log file for performance. The DataExtensions.cs class was created specifically to handle database recreation automatically when the application starts.

5. DataExtensions.cs & Data Seeding

5.1 DataExtensions.cs

To improve code organization and implement automatic database initialization, I added a new file named DataExtensions.cs inside the Data folder. This file serves two important purposes:

- Decoupling — it separates database setup logic from Program.cs, keeping the startup file clean and readable.

- Automatic Database Recreation — it ensures the database is automatically created (or re-created) each time the application starts, even if the database file has been deleted.

This file is registered in Program.cs and executes as part of the application startup sequence, providing a reliable and repeatable database initialization mechanism.

5.2 Data Seeding

Data seeding refers to the practice of inserting a predefined set of initial records into the database when the application first runs, so that it is not empty upon launch. In the GameStore.Api project, this was applied to pre-populate the Genres table with default game categories. Without data seeding, a newly deployed application would have no genres available, which would prevent games from being created through the API.

> ℹ️ **Data Seeding Analogy**
>
> Think of data seeding like stocking a new store before opening day. Just as a shop must have products on shelves before customers can browse, a database must have foundational reference data (such as genre categories) before the application can function correctly.

6. Configuring Logging Levels

I studied and configured application logging levels as part of this phase. Logging is essentially the daily diary of an application — it records what the application is doing, which requests it is processing, which errors occur, and how long operations take.

The core purpose of logging levels is to control the volume of information being recorded. An application can generate thousands of log entries per second, and recording everything would bloat log files and slow the system. By assigning severity levels, we can filter out low-priority messages in production while capturing all details during development.

Concept	Explanation
Trace	Most detailed level. Records every step. Used only for deep debugging during development.
Debug	Detailed diagnostic information. Used during development and testing phases.
Information	General operational events. Confirms that things are working as expected.
Warning	Indicates something unexpected occurred, but the application continues to function.
Error	A serious failure occurred that requires attention. Part of the request may have failed.
Critical	A severe failure that may cause the application to crash or become

Concept	Explanation
	unavailable.

Logging levels are configured in the appsettings.json file under the Logging section, and can be set differently for development and production environments using appsettings.Development.json and appsettings.Production.json respectively.

7. Dependency Injection

I learned and applied the Dependency Injection (DI) design pattern to connect the database context to my API endpoints. In .NET, services such as GameStoreContext are registered with the DI container in Program.cs. When an endpoint handler needs to interact with the database, the DI container automatically provides the correct instance of the context — the developer does not need to manually create or manage it.

This pattern makes the codebase more testable, maintainable, and loosely coupled. By injecting GameStoreContext into endpoint handlers, each handler gains direct access to the database without depending on global variables or static instances.

8. Asynchronous Programming

One of the most impactful concepts I studied in this phase was asynchronous programming using C#'s async/await pattern. The diagram below illustrates the difference between synchronous and asynchronous request handling:

Traditional (Sync)	Async / Await (.NET)
Request 1 arrives	Request 1 arrives
Wait for DB query...	DB query starts (non-blocking)
Response 1 sent	Request 2 handled meanwhile
Request 2 arrives	Request 3 handled meanwhile
Wait for DB query...	DB responds → Response 1 sent
Response 2 sent	DB responds → Response 2 sent

In a synchronous model, the server can only process one request at a time — while waiting for a database query to complete, all other incoming requests are blocked. In the asynchronous model, the server releases the thread while waiting for the database and immediately starts handling other requests. When the database responds, the result is delivered and the original response is sent.

I added async/await support to all CRUD endpoints in GameEndpoints.cs. An example of the updated POST endpoint signature is shown below:

```
group.MapPost("/", async (CreateGameDto newGame, GameStoreContext
dbContext) =>
{
    // ... create game logic
    await dbContext.SaveChangesAsync();
});
```

The async keyword on the lambda function and the await keyword on database operations (such as SaveChangesAsync, ToListAsync, FindAsync) are the two key changes that enable non-blocking execution.

9. Database-Backed CRUD Operations

After establishing the async foundation, I updated all four CRUD endpoints to read from and write to the actual SQLite database via EF Core, replacing the earlier in-memory list. The following table summarizes the updated endpoints:

HTTP Method	Route	EF Core Operation	HTTP Response
GET	/games	dbContext.Games.ToListAsync() — returns GameDto list	200 OK
GET	/games/{id}	dbContext.Games.FindAsync(id) — returns GameDetailsDto	200 OK / 404
POST	/games	dbContext.Games.Add(game) + SaveChangesAsync()	201 Created
PUT	/games/{id}	Find + modify + SaveChangesAsync()	204 No Content
DELETE	/games/{id}	Find + Remove + SaveChangesAsync()	204 No Content

An important lesson learned during this step is that any change made to a Model class (such as adding a new property) requires a new migration to be created and applied. The command sequence is: dotnet ef migrations add <MigrationName> followed by dotnet ef database update. This ensures the database schema remains in sync with the application's model definitions.

> ℹ️ **Key Understanding: GameDetailsDto vs. GameDto**
>
> I gained a clear understanding of when to use different DTOs for the same entity. For the general list endpoint (GET /games), I return a GameDto which contains a summary of each game. For the single-item endpoint (GET /games/{id}), I return a GameDetailsDto which includes additional fields such as the full genre details. This pattern avoids over-fetching data and keeps API responses lean and purposeful.

10. Genres API Endpoint

After completing the full CRUD implementation for the Games endpoint, I extended the API by creating a dedicated Genres endpoint. This followed the same pattern established for Games:

- Created GenreDto.cs inside the Dtos folder via Solution Explorer, containing Id and Name properties.
- Created a new endpoint file for genre-related routes.
- Implemented a GET /genres endpoint that retrieves all genre records from the database asynchronously.
- Tested the endpoint using the REST Client extension in VS Code with the request: GET http://localhost:5130/genres

The Genres API returned a correctly structured JSON array of genre objects, confirming that the endpoint, DTO, and database connection were all functioning correctly.

11. Updated Application Architecture

The application architecture has evolved significantly since Part I. The diagram below reflects the current state of the GameStore.Api project, incorporating all the components added during this phase:

Notable additions compared to Part I include: the DataExtensions.cs layer for database lifecycle management, the expanded DTO layer (GameDetailsDto and GenreDto), and the full persistence of all CRUD operations through EF Core to the SQLite database file.

12. Progress Summary — Part II

The table below provides a consolidated view of all topics covered in this phase of study:

Topic / Module	Key Achievement	Status
EF Core Tooling Installation	dotnet ef tool and Design package installed	✔ Completed

Topic / Module	Key Achievement	Status
Migration Generation	InitialCreate.cs generated with Games and Genres tables	✔ Completed
Database Update (Migration Apply)	dotnet ef database update executed successfully	✔ Completed
Connection String Configuration	Correct SQLite format applied in appsettings.json	✔ Completed
Environment Variable Configuration	Production.db created via $env: PowerShell command	✔ Completed
SQLite Explorer Extension	Installed and used to inspect GameStore.db visually	✔ Completed
DataExtensions.cs	Created for decoupling and auto database initialization	✔ Completed
Data Seeding	Initial Genres data seeded into database on app start	✔ Completed
Logging Level Configuration	Logging levels reviewed and configured in appsettings.json	✔ Completed
Dependency Injection	GameStoreContext injected into endpoint handlers	✔ Completed
Async/Await Implementation	All endpoints updated to async with EF Core async methods	✔ Completed
GET /games (DB-backed)	Returns GameDto list from database using ToListAsync	✔ Completed
GET /games/{id} (DB-backed)	Returns GameDetailsDto using FindAsync with 404 handling	✔ Completed
POST /games (DB-backed)	Creates record in database with SaveChangesAsync	✔ Completed
PUT /games/{id} (DB-backed)	Updates record in database with SaveChangesAsync	✔ Completed
DELETE /games/{id} (DB-backed)	Removes record from database with SaveChangesAsync	✔ Completed
Genres API — GET /genres	GenreDto created; endpoint tested and verified	✔ Completed
Frontend Integration	Initiated; continuation planned in Part III	⚠ In Progress

13. Next Steps

The following topics are planned for the next phase of study, to be documented in Part III:

- Complete the backend-to-frontend connection — integrating the GameStore API with a client-side application.
- Implement error handling middleware for consistent API error responses.
- Explore API versioning and route grouping best practices.
- Study and apply the repository pattern for further separation between data access logic and endpoint logic.
- Investigate unit testing for .NET minimal API endpoints.

14. Conclusion

This second phase of my .NET self-study has been both challenging and deeply rewarding. I have successfully transitioned the GameStore.Api project from an in-memory prototype to a fully database-backed REST API with proper connection management, asynchronous operations, data seeding, and logging configuration. These are not introductory concepts — they represent the foundations upon which real-world enterprise applications are built.

Each concept studied has direct applicability to my responsibilities as an Application Management Officer. Understanding how applications manage data, handle configuration across environments, and scale to serve many concurrent users gives me a significantly stronger foundation for evaluating, supporting, and eventually contributing to application systems within my organization.

I remain committed to continuing this learning journey and will document further progress in Part III of this report series.

Dereje Abtew

Application Management Officer (Trainee Banker)

April 30, 2026